

CENG 501 Fall 2025 Homework 3

29.12.2025

IMPORTANT COPYRIGHT NOTICE: **You are not allowed to share** this document with anybody else. For example, you should not publish it on your publicly accessible webpages or GitHub repositories.

IMPORTANT NOTE: **What you submit should be your own work!** You may use large language models (LLMs) or other AI-assisted tools for this assignment. However, **learning** the concepts and skills covered by this homework **is your responsibility**. You are expected to understand and be able to explain any code or solution you submit. By submitting your work, you acknowledge full responsibility for its content and correctness -- regardless of whether it was produced with or without AI assistance.

I may randomly call students to explain their submitted work. If a student cannot convincingly explain their code, they may receive a zero or a reduced grade.

By submitting your solution to this assignment, you agree to the METU Code of Honor:

> “As reliable, responsible and honorable individuals, all members of Middle East Technical University embrace only the success and recognition they deserve, and act with integrity in the use, evaluation and presentation of facts, data and documents.”

Good luck!

Instructor: Dr. Emre Akbas

In this homework you will implement a simple, CLIP-style vision-language model (VLM). You will use off-the-shelf, pre-trained vision and text encoders, and learn shallow projection layers to align their representations, using the SigLIP loss. During training, the vision encoder and text encoder will be used as frozen models.

To keep it simple and feasible with minimal resources, you will use the MNIST dataset. However, to keep it interesting, you will concatenate MNIST images side by side to form 3-digit numbers.

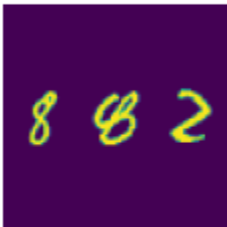
You should implement a custom data loader which randomly samples three images from the MNIST training set, and then concatenates them side-by-side. Since each MNIST image is 28x28 pixels, the concatenated version will be 28x84. Since most pre-trained ViTs use an image size of 224x224 and we want to make use of a pre-trained ViT, you should pad this 28x84 image with zeros from top and bottom, so that you obtain a square 84x84 image. You will also have to resize these 84x84 images to 224x224 before feeding them to the vision encoder. For each

such image, you should also create a caption which is simply how that number is read in English. Below you can see 4 example images along with their captions:

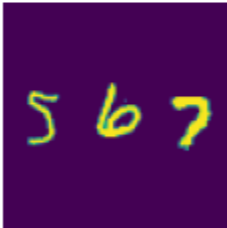
eight hundred ninety two



eight hundred eighty two



five hundred sixty seven



nine hundred nineteen



The vision encoder should be **pre-trained** ViT. I suggest you use a tiny one, for example, `vit_tiny_patch16_224`. It is available in the `timm` library, you can easily load it with:

```
import timm

model = timm.create_model(
    "vit_tiny_patch16_224",
    pretrained=True,
    num_classes=0          # removes classification head
```

```
)  
model.eval()
```

Setting “num_classes=0” removes the classification head from the model and allows you to get the image embedding, which is 192 dimensional for this model. You are free to use any other vision encoder but justify why you chose it in your report.

For the text encoder, you should again use a lightweight model. For example, you can use `distilbert-base-uncased`, which is available from HuggingFace. You need to obtain the embedding for a whole caption. There are a few ways to do this. You can use the embedding of the first token, which is usually used by BERT models as the classification token. Using mean pooling over tokens is also acceptable. The final embedding dimension for a caption is 768 dimensional, if you use the recommended `distilbert-base-uncased` model.

Next, you need to align the vision embedding and text embedding into a common 128 dimensional space. For this, you need to create two linear layers, one for projecting the vision embedding (from 192 to 128) and the other for projecting the text embedding (from 768 to 128).

You should train these two linear layers using the SigLIP loss:

$$-\frac{1}{|\mathcal{B}|} \sum_{i=1}^{|\mathcal{B}|} \sum_{j=1}^{|\mathcal{B}|} \underbrace{\log \frac{1}{1 + e^{z_{ij}(-t\mathbf{x}_i \cdot \mathbf{y}_j + b)}}}_{\mathcal{L}_{ij}} \quad \text{(Equation on page 3 in } \underline{\a href="https://arxiv.org/abs/2303.15343"}{https://arxiv.org/abs/2303.15343})$$

An implementation example (Algorithm 1) is given on page 2 of the paper.

You have two tasks in this homework.

Task 1: Implement the VLM described above and train it. During training, never show the number “123” to the model. In other words, the number 123 should not be in the training set.

Task 2: Write a function to evaluate your model. This evaluator function should take an 84x84 image and compute its similarity to all 1000 captions (numbers from 000 to 999). For efficiency, you should compute the embeddings of all captions once and then reuse them.

- Show a few example evaluations.
- Report top-1 and top-5 accuracy for a test set of at least 1000 images.
- Then, compute the top-1 and top-5 accuracies for “123” images. They were not part of the training set.

Submit a Jupyter Notebook file (ipynb) with all your results in it.